Iteration G1: Capturing an Order

An order is a set of line items, along with details of the purchase transaction. Our cart already contains `line_items`, so all we need to do is add an `order_id` column to the `line_items` table and create an `orders` table based on the Initial guess at application data diagram, combined with a brief chat with our customer.

First we create the order model and update the `line_items` table:

```
depot> bin/rails generate scaffold Order name address:text email \
         pay_type:integer
depot> bin/rails generate migration add_order_to_line_item order:references
```

Note that we didn't specify any data type for two of the four columns. This is because the data type defaults to `string`. This is yet another small way in which Rails makes things easier for you in the most common case without making things any more cumbersome when you need to specify a data type.

Also note that we defined `pay_type` as an integer. While this is an efficient way to store data that can only store discrete values, storing data in this way requires keeping track of which values are used for which payment type. Rails can do this for you through the use of `enum` declarations placed in the model class. Add this code to `app/models/order.rb`:

rails7/depot_o/app/models/order.rb

```
class Order < ApplicationRecord
  enum pay_type: {
    "Check"          => 0,
    "Credit card"    => 1,
    "Purchase order" => 2
  }
end
```

Finally, we need to modify the second migration to indicate that `cart_id` can be null in records. This is done by modifying the existing `add_reference` line to say `null: true` and adding a new `change_column` line to enable nulls in the `cart_id`